

(https:// /  Q...



HackMD

(https://hackmd.io?utm_source=view-page&utm_medium=logo-nav)

Quiz1 of Computer Architecture (2025 Fall)

Solutions

Problem A

Let's implement a small vector API that stores arbitrary `void *` values. It makes no assumptions about what the pointers reference or who owns them. If an element points to heap memory, you are responsible for freeing it. The API centers on simple push/pop semantics plus callbacks for processing and destroying elements. The test suite ([Appendix A \(https://gist.github.com/jserv/7d79fae597b0c15adcb8877781caa6d8\)](https://gist.github.com/jserv/7d79fae597b0c15adcb8877781caa6d8)) reports:

```
=== Running Vector Tests ===
Testing vector_init... PASSED
Testing vector_push and vector_pop... PASSED
Testing vector_get_at... PASSED
Testing vector_delete_at... PASSED
Testing vector_for_each... PASSED
Testing vector_delete_all... PASSED
Testing vector automatic resize... PASSED
Testing vector_get_end... PASSED
Testing vector_set_at... PASSED

=== All Tests Passed ===
```

No assertions should fire. Below is a complete code snippet consistent with those tests. Complete the code snippet below to ensure it works as expected.

Notes:

- Ownership is explicit: `vector_pop` returns the stored pointer without freeing it; `vector_delete_at` and `vector_delete_all` invoke the optional destructor.
- `vector_get_end` returns the one-past-the-last index (i.e., current length), which aligns with typical “end” semantics.
- Growth is automatic and amortized; no assertions are used.

```

#include <assert.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define VECTOR_MIN_SIZE 16

typedef struct {
    void **data;
    size_t size;      /* Allocated size */
    size_t count;     /* Number of elements */
    size_t free_slot; /* Index of a known hole */
} vector_t;

typedef void (*vector_delete_callback_t)(void *);
typedef void (*vector_foreach_callback_t)(void *, void *);

/* Initialize vector */
void vector_init(vector_t *v)
{
    v->data = NULL;
    v->size = 0;
    v->count = 0;
    v->free_slot = 0;
}

/* Push element to vector, returns index */
int32_t vector_push(vector_t *v, void *ptr)
{
    if (!v->size) {
        v->size = VECTOR_MIN_SIZE;
        v->data = calloc(v->size, sizeof(void *));
    }

    /* Reuse free slot if available */
    if (v->free_slot && v->free_slot < v->count) {
        size_t idx = v->free_slot;
        v->data[idx] = ptr;
        v->free_slot = 0;
        return idx;
    }

    /* Resize if needed */
    if (v->count == v->size) {
        v->size *= 2;
        v->data = realloc(v->data, v->size * sizeof(void *));
        memset(v->data + v->count, 0, (v->size - v->count) * sizeof(void *));
    }

    v->data[v->count] = ptr;
    return v->count++;
}

/* Pop last element */
void *vector_pop(vector_t *v)

```

```

{
    if (!v->count)
        return NULL;
    void *last = v->data[--v->count];
    v->data[v->count] = NULL;
    return last;
}

/* Get element at index */
void *vector_get_at(vector_t *v, size_t index)
{
    return (index < v->count) ? v->data[index] : NULL;
}

/* Set element at index (replaces existing) */
void *vector_set_at(vector_t *v, size_t index, void *ptr)
{
    if (index >= v->count)
        return NULL;
    v->data[index] = ptr;
    return ptr;
}

/* Get last element */
void *vector_get_end(vector_t *v)
{
    return v->count ? v->data[v->count - 1] : NULL;
}

/* Delete at index (creates hole) */
void vector_delete_at(vector_t *v, size_t index)
{
    if (index < v->count) {
        v->data[index] = NULL;
        v->free_slot = index;
    }
}

/* Iterate over elements */
void *vector_for_each(vector_t *v, vector_foreach_callback_t cb, void *data)
{
    if (!cb)
        return NULL;

    for (size_t i = 0; i < v->count; i++) {
        if (!v->data[i])
            continue;
        void *ret = cb(v->data[i], data);
        if (ret)
            return ret;
    }
    return NULL;
}

/* Delete all elements */
void vector_delete_all(vector_t *v, vector_delete_callback_t dc)
{

```

```

        for (void *p; (p = vector_pop(v));) {
            if (dc)
                dc(p);
        }
    }

    /* Free vector memory */
    void vector_free(vector_t *v)
    {
        if (!v->data)
            return;

        free(v->data);
        v->data = NULL;
        v->size = 0;
        v->count = 0;
        v->free_slot = 0;
    }

    /* Get number of elements */
    size_t vector_used(vector_t *v)
    {
        return v->count;
    }

```

A01: `calloc(v->size, sizeof(void *))`

A02: `v->free_slot`

A03: `realloc(v->data, v->size * sizeof(void *))`

A04: `v->data[--v->count]`

A05: `v->data[i]`

A06: `cb(v->data[i], data)`

A07: `p = vector_pop(v)`

A08: `dc(p)`

A09: 至少 40 個英文單詞構成的回覆，要有明確的 C 程式碼和提及實作上的好處

Problem B

Assume uf8 implements a logarithmic 8-bit codec that maps 20-bit unsigned integers ($[0, 1,015,792]$) to 8-bit symbols via logarithmic quantization, delivering 2.5:1 compression and $\leq 6.25\%$ relative error.

The uf8 encoding scheme is ideal for sensor data applications such as temperature and distance measurements where range is more important than precision. In graphics applications, it efficiently represents level-of-detail (LOD) distances and fog density values. The scheme is also well-suited for timer implementations that use exponential backoff strategies. However, the uf8 encoding should not be used for financial

calculations where exact precision is required and rounding errors are unacceptable. It is inappropriate for cryptographic applications that require uniform distribution of values for security purposes.

☐ Decoding

$$D(b) = m \cdot 2^e + (2^e - 1) \cdot 16$$

Where $e = \lfloor b/16 \rfloor$ and $m = b \bmod 16$

☐ Encoding

$$E(v) = \begin{cases} v, & \text{if } v < 16 \\ 16e + \lfloor (v - \text{offset}(e))/2^e \rfloor, & \text{otherwise} \end{cases}$$

Where $\text{offset}(e) = (2^e - 1) \cdot 16$

Error Analysis

- Absolute Error: $\Delta_{\max} = 2^e - 1$
- Relative Error: $\varepsilon_{\max} = 1/16 = 6.25\%$
- Expected Error: $\mathbb{E}[\varepsilon] \approx 3\%$

Information Theory

- Input Entropy: 20 bits
- Output Entropy: 8 bits
- Theoretical Minimum: 7.6 bits (for 6.25% error bound)
- Efficiency: $8/7.6 = 95\%$ optimal

Exponent	Range	Step Size
0	[0, 15]	1
1	[16, 46]	2
2	[48, 108]	4
3	[112, 232]	8
...	...	2^e
15	[524,272, 1,015,792]	32,768

```

#include <stdbool.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>

typedef uint8_t uf8;

static inline unsigned clz(uint32_t x)
{
    int n = 32, c = 16;
    do {
        uint32_t y = x >> c;
        if (y) {
            n -= c;
            x = y;
        }
        c >>= 1;
    } while (c);
    return n - x;
}

/* Decode uf8 to uint32_t */
uint32_t uf8_decode(uf8 f1)
{
    uint32_t mantissa = f1 & 0x0f;
    uint8_t exponent = f1 >> 4;
    uint32_t offset = (0x7FFF >> (15 - exponent)) << 4;
    return (mantissa << exponent) + offset;
}

/* Encode uint32_t to uf8 */
uf8 uf8_encode(uint32_t value)
{
    /* Use CLZ for fast exponent calculation */
    if (value < 16)
        return value;

    /* Find appropriate exponent using CLZ hint */
    int lz = clz(value);
    int msb = 31 - lz;

    /* Start from a good initial guess */
    uint8_t exponent = 0;
    uint32_t overflow = 0;

    if (msb >= 5) {
        /* Estimate exponent - the formula is empirical */
        exponent = msb - 4;
        if (exponent > 15)
            exponent = 15;

        /* Calculate overflow for estimated exponent */
        for (uint8_t e = 0; e < exponent; e++)
            overflow = (overflow << 1) + 16;

        /* Adjust if estimate was off */
    }
}

```

```

        while (exponent > 0 && value < overflow) {
            overflow = (overflow - 16) >> 1;
            exponent--;
        }

    }

    /* Find exact exponent */
    while (exponent < 15) {
        uint32_t next_overflow = (overflow << 1) + 16;
        if (value < next_overflow)
            break;
        overflow = next_overflow;
        exponent++;
    }

    uint8_t mantissa = (value - overflow) >> exponent;
    return (exponent << 4) | mantissa;
}

/* Test encode/decode round-trip */
static bool test(void)
{
    int32_t previous_value = -1;
    bool passed = true;

    for (int i = 0; i < 256; i++) {
        uint8_t f1 = i;
        int32_t value = uf8_decode(f1);
        uint8_t f12 = uf8_encode(value);

        if (f1 != f12) {
            printf("%02x: produces value %d but encodes back to %02x\n", f1,
                value, f12);
            passed = false;
        }

        if (value <= previous_value) {
            printf("%02x: value %d <= previous_value %d\n", f1, value,
                previous_value);
            passed = false;
        }

        previous_value = value;
    }

    return passed;
}

int main(void)
{
    if (test()) {
        printf("All tests passed.\n");
        return 0;
    }
    return 1;
}

```

B01: 15 - exponent

B02: 4

B03: mantissa << exponent

B04: overflow - 16

B05: overflow << 1

B06: (value - overflow) >> exponent

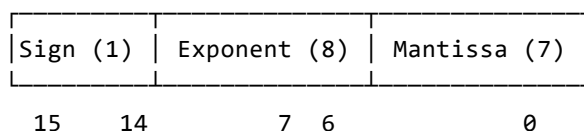
B07: exponent << 4

B08: 至少 40 個英文單詞構成的回覆，要有明確的 C 程式碼和提及實作上的好處

Problem C

The bfloat16 format (16-bit, from Google Brain) preserves float32's dynamic range by keeping the same 8-bit exponent, but reduces precision to a 7-bit significand (vs. 23).

Bit Layout



S: Sign bit (0 = positive, 1 = negative)

E: Exponent bits (8 bits, bias = 127)

M: Mantissa/fraction bits (7 bits)

The value v of a BFloat16 number is calculated as:

$$v = (-1)^S \times 2^{E-127} \times \left(1 + \frac{M}{128}\right)$$

where:

- $S \in \{0, 1\}$ is the sign bit
- $E \in [1, 254]$ is the biased exponent
- $M \in [0, 127]$ is the mantissa value

Special Cases

- Zero: $E = 0, M = 0 \Rightarrow v = (-1)^S \times 0$
- Infinity: $E = 255, M = 0 \Rightarrow v = (-1)^S \times \infty$
- NaN: $E = 255, M \neq 0 \Rightarrow v = \text{NaN}$
- Denormals: Not supported (flush to zero)

Complete the following code to implement correct conversions, arithmetic, and rounding behavior.

```

#include <stdbool.h>
#include <stdint.h>
#include <string.h>

typedef struct {
    uint16_t bits;
} bf16_t;

#define BF16_SIGN_MASK 0x8000U
#define BF16_EXP_MASK 0x7F80U
#define BF16_MANT_MASK 0x007FU
#define BF16_EXP_BIAS 127

#define BF16_NAN() ((bf16_t) {.bits = 0x7FC0})
#define BF16_ZERO() ((bf16_t) {.bits = 0x0000})

static inline bool bf16_isnan(bf16_t a)
{
    return ((a.bits & BF16_EXP_MASK) == BF16_EXP_MASK) &&
        (a.bits & BF16_MANT_MASK);
}

static inline bool bf16_isinf(bf16_t a)
{
    return ((a.bits & BF16_EXP_MASK) == BF16_EXP_MASK) &&
        !(a.bits & BF16_MANT_MASK);
}

static inline bool bf16_iszero(bf16_t a)
{
    return !(a.bits & 0x7FFF);
}

static inline bf16_t f32_to_bf16(float val)
{
    uint32_t f32bits;
    memcpy(&f32bits, &val, sizeof(float));
    if (((f32bits >> 23) & 0xFF) == 0xFF)
        return (bf16_t) {.bits = (f32bits >> 16) & 0xFFFF};
    f32bits += ((f32bits >> 16) & 1) + 0x7FFF;
    return (bf16_t) {.bits = f32bits >> 16};
}

static inline float bf16_to_f32(bf16_t val)
{
    uint32_t f32bits = ((uint32_t) val.bits) << 16;
    float result;
    memcpy(&result, &f32bits, sizeof(float));
    return result;
}

static inline bf16_t bf16_add(bf16_t a, bf16_t b)
{
    uint16_t sign_a = (a.bits >> 15) & 1;
    uint16_t sign_b = (b.bits >> 15) & 1;
    int16_t exp_a = ((a.bits >> 7) & 0xFF);

```

```

int16_t exp_b = ((b.bits >> 7) & 0xFF);
uint16_t mant_a = a.bits & 0x7F;
uint16_t mant_b = b.bits & 0x7F;

if (exp_a == 0xFF) {
    if (mant_a)
        return a;
    if (exp_b == 0xFF)
        return (mant_b || sign_a == sign_b) ? b : BF16_NAN();
    return a;
}
if (exp_b == 0xFF)
    return b;
if (!exp_a && !mant_a)
    return b;
if (!exp_b && !mant_b)
    return a;
if (exp_a)
    mant_a |= 0x80;
if (exp_b)
    mant_b |= 0x80;

int16_t exp_diff = exp_a - exp_b;
uint16_t result_sign;
int16_t result_exp;
uint32_t result_mant;

if (exp_diff > 0) {
    result_exp = exp_a;
    if (exp_diff > 8)
        return a;
    mant_b >>= exp_diff;
} else if (exp_diff < 0) {
    result_exp = exp_b;
    if (exp_diff < -8)
        return b;
    mant_a >>= -exp_diff;
} else {
    result_exp = exp_a;
}

if (sign_a == sign_b) {
    result_sign = sign_a;
    result_mant = (uint32_t) mant_a + mant_b;

    if (result_mant & 0x100) {
        result_mant >>= 1;
        if (++result_exp >= 0xFF)
            return (bf16_t) {.bits = (result_sign << 15) | 0x7F80};
    }
} else {
    if (mant_a >= mant_b) {
        result_sign = sign_a;
        result_mant = mant_a - mant_b;
    } else {
        result_sign = sign_b;
        result_mant = mant_b - mant_a;
    }
}

```

```

    }

    if (!result_mant)
        return BF16_ZERO();
    while (!(result_mant & 0x80)) {
        result_mant <<= 1;
        if (--result_exp <= 0)
            return BF16_ZERO();
    }
}

return (bf16_t) {
    .bits = (result_sign << 15) | ((result_exp & 0xFF) << 7) |
        (result_mant & 0x7F),
};
}

static inline bf16_t bf16_sub(bf16_t a, bf16_t b)
{
    b.bits ^= BF16_SIGN_MASK;
    return bf16_add(a, b);
}

static inline bf16_t bf16_mul(bf16_t a, bf16_t b)
{
    uint16_t sign_a = (a.bits >> 15) & 1;
    uint16_t sign_b = (b.bits >> 15) & 1;
    int16_t exp_a = ((a.bits >> 7) & 0xFF);
    int16_t exp_b = ((b.bits >> 7) & 0xFF);
    uint16_t mant_a = a.bits & 0x7F;
    uint16_t mant_b = b.bits & 0x7F;

    uint16_t result_sign = sign_a ^ sign_b;

    if (exp_a == 0xFF) {
        if (mant_a)
            return a;
        if (!exp_b && !mant_b)
            return BF16_NAN();
        return (bf16_t) {.bits = (result_sign << 15) | 0x7F80};
    }
    if (exp_b == 0xFF) {
        if (mant_b)
            return b;
        if (!exp_a && !mant_a)
            return BF16_NAN();
        return (bf16_t) {.bits = (result_sign << 15) | 0x7F80};
    }
    if ((!exp_a && !mant_a) || (!exp_b && !mant_b))
        return (bf16_t) {.bits = result_sign << 15};

    int16_t exp_adjust = 0;
    if (!exp_a) {
        while (!(mant_a & 0x80)) {
            mant_a <<= 1;
            exp_adjust--;
        }
    }

```

```

        exp_a = 1;
    } else
        mant_a |= 0x80;
    if (!exp_b) {
        while (!(mant_b & 0x80)) {
            mant_b <<= 1;
            exp_adjust--;
        }
        exp_b = 1;
    } else
        mant_b |= 0x80;

    uint32_t result_mant = (uint32_t) mant_a * mant_b;

    int32_t result_exp = (int32_t) exp_a + exp_b - BF16_EXP_BIAS + exp_adjust;

    if (result_mant & 0x8000) {
        result_mant = (result_mant >> 8) & 0x7F;
        result_exp++;
    } else
        result_mant = (result_mant >> 7) & 0x7F;

    if (result_exp >= 0xFF)
        return (bf16_t) {.bits = (result_sign << 15) | 0x7F80};
    if (result_exp <= 0) {
        if (result_exp < -6)
            return (bf16_t) {.bits = result_sign << 15};
        result_mant >>= (1 - result_exp);
        result_exp = 0;
    }

    return (bf16_t) {.bits = (result_sign << 15) | ((result_exp & 0xFF) << 7) |
        (result_mant & 0x7F)};
}

static inline bf16_t bf16_div(bf16_t a, bf16_t b)
{
    uint16_t sign_a = (a.bits >> 15) & 1;
    uint16_t sign_b = (b.bits >> 15) & 1;
    int16_t exp_a = ((a.bits >> 7) & 0xFF);
    int16_t exp_b = ((b.bits >> 7) & 0xFF);
    uint16_t mant_a = a.bits & 0x7F;
    uint16_t mant_b = b.bits & 0x7F;

    uint16_t result_sign = sign_a ^ sign_b;

    if (exp_b == 0xFF) {
        if (mant_b)
            return b;
        /* Inf/Inf = NaN */
        if (exp_a == 0xFF && !mant_a)
            return BF16_NAN();
        return (bf16_t) {.bits = result_sign << 15};
    }
    if (!exp_b && !mant_b) {
        if (!exp_a && !mant_a)
            return BF16_NAN();

```

```

        return (bf16_t) {.bits = (result_sign << 15) | 0x7F80};
    }
    if (exp_a == 0xFF) {
        if (mant_a)
            return a;
        return (bf16_t) {.bits = (result_sign << 15) | 0x7F80};
    }
    if (!exp_a && !mant_a)
        return (bf16_t) {.bits = result_sign << 15};

    if (exp_a)
        mant_a |= 0x80;
    if (exp_b)
        mant_b |= 0x80;

    uint32_t dividend = (uint32_t) mant_a << 15;
    uint32_t divisor = mant_b;
    uint32_t quotient = 0;

    for (int i = 0; i < 16; i++) {
        quotient <= 1;
        if (dividend >= (divisor << (15 - i))) {
            dividend -= (divisor << (15 - i));
            quotient |= 1;
        }
    }

    int32_t result_exp = (int32_t) exp_a - exp_b + BF16_EXP_BIAS;

    if (!exp_a)
        result_exp--;
    if (!exp_b)
        result_exp++;

    if (quotient & 0x8000)
        quotient >>= 8;
    else {
        while (!(quotient & 0x8000) && result_exp > 1) {
            quotient <= 1;
            result_exp--;
        }
        quotient >>= 8;
    }
    quotient &= 0x7F;

    if (result_exp >= 0xFF)
        return (bf16_t) {.bits = (result_sign << 15) | 0x7F80};
    if (result_exp <= 0)
        return (bf16_t) {.bits = result_sign << 15};
    return (bf16_t) {.bits = (result_sign << 15) | ((result_exp & 0xFF) << 7) |
        (quotient & 0x7F)};
}

```

C01: f32bits >> 23

C02: f32bits >> 16

C03: f32bits >> 16

C04: 0x7FFF

C05: f32bits >> 16

C06: 16

C07: 0x7F80

C08: 0x80

C09: 0x7F

C10: 0x8000

C11: 0x100

Square Root

$$\sqrt{a} = \sqrt{2^{e_a} \times m_a} = 2^{e_a/2} \times \sqrt{m_a}$$

The `bf16_sqrt` implementation uses pure integer arithmetic without floating-point library dependencies:

1. Special Case Handling (IEEE-754 compliant):

- $\sqrt{+0} = +0$
- $\sqrt{-0} = 0$
- $\sqrt{+\infty} = +\infty$
- $\sqrt{-\infty} = \text{NaN}$
- $\sqrt{\text{NaN}} = \text{NaN}$
- $\sqrt{x} = \text{NaN}$ for all $x < 0$

2. Exponent Processing:

For even exponents (e_a is even):

$$e_r = \frac{e_a}{2}, \quad m' = m_a$$

For odd exponents (e_a is odd):

$$e_r = \frac{e_a - 1}{2}, \quad m' = 2 \times m_a$$

This ensures the mantissa remains in the normalized range $[1, 2)$ or $[2, 4)$ for processing.

3. Mantissa Square Root via Binary Search:

The algorithm finds r such that $r^2 \approx m' \times 128$ using binary search. The scaling factor 128 represents the implicit 1.0 in the mantissa representation.

4. **Normalization:**

- Ensure result mantissa is in range $[128, 256)$
- Adjust exponent if necessary
- Extract 7-bit mantissa (removing implicit 1)

Mathematical Properties:

- Monotonic: $x < y \Rightarrow \sqrt{x} \leq \sqrt{y}$
- Identity: $\sqrt{x^2} = |x|$ for $x \geq 0$
- Precision: Maximum relative error $< 0.5\%$ across all BF16 range

```

static inline bf16_t bf16_sqrt(bf16_t a)
{
    uint16_t sign = (a.bits >> 15) & 1;
    int16_t exp = ((a.bits >> 7) & 0xFF);
    uint16_t mant = a.bits & 0x7F;

    /* Handle special cases */
    if (exp == 0xFF) {
        if (mant)
            return a; /* NaN propagation */
        if (sign)
            return BF16_NAN(); /* sqrt(-Inf) = NaN */
        return a; /* sqrt(+Inf) = +Inf */
    }

    /* sqrt(0) = 0 (handle both +0 and -0) */
    if (!exp && !mant)
        return BF16_ZERO();

    /* sqrt of negative number is NaN */
    if (sign)
        return BF16_NAN();

    /* Flush denormals to zero */
    if (!exp)
        return BF16_ZERO();

    /* Direct bit manipulation square root algorithm */
    /* For sqrt: new_exp = (old_exp - bias) / 2 + bias */
    int32_t e = exp - BF16_EXP_BIAS;
    int32_t new_exp;

    /* Get full mantissa with implicit 1 */
    uint32_t m = 0x80 | mant; /* Range [128, 256) representing [1.0, 2.0) */

    /* Adjust for odd exponents: sqrt(2^odd * m) = 2^((odd-1)/2) * sqrt(2*m) */
    if (e & 1) {
        m <<= 1; /* Double mantissa for odd exponent */
        new_exp = ((e - 1) >> 1) + BF16_EXP_BIAS;
    } else {
        new_exp = (e >> 1) + BF16_EXP_BIAS;
    }

    /* Now m is in range [128, 256) or [256, 512) if exponent was odd */
    /* Binary search for integer square root */
    /* We want result where result^2 = m * 128 (since 128 represents 1.0) */

    uint32_t low = 90; /* Min sqrt (roughly sqrt(128)) */
    uint32_t high = 256; /* Max sqrt (roughly sqrt(512)) */
    uint32_t result = 128; /* Default */

    /* Binary search for square root of m */
    while (low <= high) {
        uint32_t mid = (low + high) >> 1;
        uint32_t sq = (mid * mid) / 128; /* Square and scale */

```

```

    if (sq <= m) {
        result = mid; /* This could be our answer */
        low = mid + 1;
    } else {
        high = mid - 1;
    }
}

/* result now contains sqrt(m) * sqrt(128) / sqrt(128) = sqrt(m) */
/* But we need to adjust the scale */
/* Since m is scaled where 128=1.0, result should also be scaled same way */

/* Normalize to ensure result is in [128, 256) */
if (result >= 256) {
    result >>= 1;
    new_exp++;
} else if (result < 128) {
    while (result < 128 && new_exp > 1) {
        result <<= 1;
        new_exp--;
    }
}

/* Extract 7-bit mantissa (remove implicit 1) */
uint16_t new_mant = result & 0x7F;

/* Check for overflow/underflow */
if (new_exp >= 0xFF)
    return (bf16_t) {.bits = 0x7F80}; /* +Inf */
if (new_exp <= 0)
    return BF16_ZERO();

return (bf16_t) {.bits = ((new_exp & 0xFF) << 7) | new_mant};
}

```

C12: 90

C13: 256

C14: 256

C15: 0x7F

C16: 0xFF

C17: 0x7F80

C18: 0xFF

C19: 7

C20: 0.3883%

C21: 達 30 words 就給分

The maximum relative error of 0.3883% occurs at the smallest normalized values (near 1.18×10^{-38}) where quantization effects are most pronounced. The error distribution is remarkably uniform across 76 orders of magnitude, with consistent mean error of

~0.15% from 10^{-38} to 10^{38} .